



VerifiKar

PROJECT SUPERVISOR

Mr. S. Farooq Zaidi

PROJECT TEAM

Jaswant Lal	22K-4473
Muhammad Huzaifa	22K-4641
Usman Tanveer	22K-4478

Submitted in partial fulfillment of the requirements for the degree of
Bachelor of Science in Computer Science

FAST SCHOOL OF COMPUTING
NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES
Karachi Campus
Spring 2026

Project Supervisor	Mr. S. Farooq Zaidi	
Project Team	Jaswant Lal	22K-4473
	Muhammad Huzaifa	22K-4641
	Usman Tanveer	22K-4478
Submission Date	May 10, 2026	

Mr. S. Farooq Zaidi _____
Supervisor

FAST SCHOOL OF COMPUTING
NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES
Karachi Campus
Spring 2026

ABSTRACT

VerifiKar is an AI-powered platform for crowdsourced incident reporting and verification, built to address the shortage of reliable, hyperlocal news in Karachi. Citizens submit incident reports; comprising text, images, geolocation, and optionally video; through a React Native mobile application. The platform processes each submission through a six-stage asynchronous background pipeline: the Mistral:7B cleans and normalises text (including Roman-Urdu), the Ateeqq/ai-vs-human-image-detector model detects AI-generated or manipulated media, CLIP (ViT-B/32) and nomic-embed-text generate multimodal embeddings, a composite credibility score is computed, pgvector cosine similarity groups related reports into event clusters, and Ollama produces a concise summary published as a public post.

The system is built on a FastAPI backend, with PostgreSQL extended by PostGIS for geospatial queries and pgvector for semantic similarity search. Media is stored on Cloudflare R2, processing jobs are queued through Redis using the ARQ task queue, and push notifications are delivered via Firebase Cloud Messaging. The React Native frontend provides a location-aware feed, a Discover screen, personalised recommendations, and a user reputation system weighted by contributor trust scores.

Evaluation on a held-out sample shows 74% agreement with human credibility judgements (exceeding the 70% target), average within-cluster similarity of 0.81 versus between-cluster similarity of 0.31, and supervisor-assessed factual accuracy acceptable for 8 out of 10 generated summaries. API response times remain within interactive latency thresholds under simulated concurrent load. VerifiKar demonstrates that a fully automated end-to-end pipeline can meaningfully reduce misinformation, consolidate duplicate reports, and deliver structured local news to citizens in near real-time.

TABLE OF CONTENT

1. Introduction.....	1
1.1. Background.....	1
1.2. Problem Statement.....	1
1.3. Objectives.....	2
1.4. Scope.....	3
1.4.1. Functionalities Included.....	3
1.4.2. Excluded Functionalities.....	3
1.5. Overview of Solution.....	4
2. Related Work.....	6
2.1. Existing Systems.....	6
2.3. Limitations of Existing Work.....	8
3. Requirements.....	9
3.1. Functional Requirements.....	9
3.2. Non-Functional Requirements.....	10
3.3. System Constraints.....	11
3.4. Assumptions and Dependencies.....	11
3.4.1. Prerequisites.....	11
3.4.2. External Dependencies.....	12
4. Design.....	13
4.1. System Architecture.....	13
4.2. Software Architecture.....	13
4.2.1. Presentation Layer.....	13
4.3. Data Flow Design.....	14
4.4. Database Design.....	16
4.5. Processing Pipeline Design.....	20
4.6. Sequence Diagram.....	21
4.7. State Design.....	21
5.1. Technologies Used.....	23
5.1.1. Frontend (Mobile App).....	23
5.1.2. Backend (Application Server).....	23
5.1.3. AI and Processing Layer.....	23
5.1.4. Database and Storage.....	24
5.2. Module Description.....	24
5.3. Design of APIs.....	25
5.4. Database Design and Implementation.....	26
5.4.1. Key Implementation Features.....	26
5.4.2. Core Data Groupings.....	26

5.5. Deploying the AI Pipeline.....	27
5.6. Frontend Implementation	27
5.7. Asynchronous Processing and Parallelism.....	34
5.8. Implementing Security	34
6. Testing and Assessment	35
6.1. Testing Strategy	35
6.1.1. Unit Testing	35
6.1.2. Integration Testing	35
6.1.3. System Testing	35
6.1.4. Performance Testing	36
6.2. Test Cases	36
6.3. Results	37
6.3.1. API Response Time	37
6.3.2. System Load Performance	38
6.4. Evaluation Metrics	38
6.4.1. Clustering Accuracy.....	38
6.4.2. Credibility Scoring Accuracy.....	38
6.4.3. Summarisation Fidelity	39
6.4.4. System Efficiency	39
6.5. Observation	40
6.6. Summary of Results	40
7. Conclusion	41
7.1. Summary	41
7.2. Limitations	41
7.3. Future Work	42
8. References.....	43

1. Introduction

1.1. Background

Karachi is one of the world's most densely populated cities, in a sprawled out metropolitan area of more than 20 million people. Even so, the city does not have a particular digital platform to report emergencies hyperlocal, in real time. The neighborhood needs to be covered at neighborhood level, at neighborhood speed, with their granulates, from traditional media organizations: road accidents, fire, power outages, waterlogging, criminal incidents. Newspapers or TV channels have a slow production cycle so they are not suitable for neighborhood news, and they focus on city-wide or national stories.

Social media like Twitter, Facebook and WhatsApp groups have become a viable alternative to local news for citizens. Channels include immediacy but the information in these channels is unstructured, inconsistent, and unverifiable. The same event can produce dozens of conflicting reports – on multiple platforms – without any system to determine which reports are valid or any means to consolidate them into one coherent summary. As a consequence, misinformation is spread and accurate situational awareness is hard to find.

With the recent progress in natural language processing, multimodal machine learning, vector databases and cloud-native task queues, it is now technically possible to develop systems capable of evaluating, clustering and summarizing crowdsourced information in nearly real time. It was conceived to design such technologies for the unique environment of Karachi, by which VerifiKar is able to take raw citizen reports, validate an authenticity of those reports based on AI technologies, organize these reports into a relatedly perceivable trend and publish credible and human readable summary on a geospatial feed.

1.2. Problem Statement

- There are no exclusive facilities for hyperlocal, crowdsourced incident reporting in Karachi. There are currently no systematic ways to automatically determine the trustworthiness of information that citizens submit, or to address the duplication of information from the same event, or to streamline massed reports of the same event to a more readable, single entry of the update. The main challenges that need to be overcome are:
- Reports are not well consolidated, are not formally structured, or are reported inconsistently across platforms meaning that there is no ability to recognize “related” reports.
- Automated verification mechanisms for media and text content published by users do not exist.
- A manual process to identify multiple reports regarding an incident, and to remove the duplications of effort does not scale.
- Citizens and officials are not fully confident in a centralized, reliable information sharing for structured local events.

To create the solution, a system will need to be set up to collect original user reports, automatically recognize if they were credible, group them up by similar content, and generate structured, credible, and timely reports.

1.3. Objectives

The primary objective of this project is to design and develop an AI-powered platform for reliable crowdsourced reporting and verification of local events. The specific objectives are:

- To allow users to submit incident reports with text, media, and geographic location through an intuitive mobile interface.
- To apply automated preprocessing and cleaning of user-generated content, including Roman-Urdu normalization and profanity filtering.
- To apply AI-driven multimodal verification techniques for assessing the credibility of both text and media submissions.

- To cluster semantically, spatially, and temporally related reports into coherent event groups.
- To generate concise, well-structured incident summaries for each event cluster with provenance links to source reports.
- To deliver a real-time geospatial feed of verified local updates to registered users, with personalized recommendations.

1.4. Scope

The scope of the system defines which functionalities are included and which are explicitly excluded from the current implementation.

1.4.1. Functionalities Included

- User registration and JWT-based authentication on both Android and iOS.
- Incident reporting via a mobile interface supporting text (up to 2,000 characters), up to five images, up to two videos, and automatic geolocation tagging.
- AI-automated report processing and verification through the six-stage ARQ background pipeline.
- Semantic, spatial, and temporal clustering of related reports using pgvector cosine similarity and PostGIS spatial queries.
- AI-generated post creation from verified clusters with Ollama-produced summaries and provenance links.
- A location-aware real-time feed with filters for category, radius, credibility threshold, and recency.
- A Discover screen aggregating posts by topic, location, and event.
- Personalized post recommendations based on user interaction history and location preferences.
- Push notifications via Firebase Cloud Messaging for nearby events.

1.4.2. Excluded Functionalities

- Manual or human-moderated report validation (the system uses automated thresholds with escalation flags only).
- Geographic coverage outside the Karachi metropolitan area.
- Integration with legacy news agencies or government emergency services APIs.
- Offline mode; the system requires a continuous internet connection.

1.5. Overview of Solution

VerifiKar is a crowdsourcing infrastructural solution to generate reliable, organized news posts safely from users' reports. Users report incidents through a mobile application you write yourself in React Native that asks them for descriptive text, using an optional image or video and adding a location. The submission is reported back immediately by the FastAPI backend which writes the original submission to PostgreSQL and enqueues an ARQ asynchronous background process to run to produce the processed report.

The pipeline used in the background performs the following steps: 1. Cleans and normalises the text using the Mistral:7B, 2. Using the model "nomic-embed-text" (via Ollama), generates semantic text embedding that the image embedding is saved to pgvector, 3. Performs six image text forgery scores using the model "Ateeqq/ai-vs-human-image-detector", 4. Computes a composite credibility score from the text authenticity score, media forgery scores and cross-modal text-image consistency, 5. Computes pgvector cosine distance between the embedding and existing cluster centroids, and constructs a new one if there is no centroid that matches the text sufficiently, 6. Publishes a summary created by the Mistral:7B as a post linked to the cluster.

Processed posts are surfaced via location-aware feed, Discover screen, and a personalised recommendation engine. Community interactions, upvotes/downvotes/flags/comments etc., will further improve the credibility metrics and surface the most trustworthy content. What you get in the end is a framework that translates unstructured, noisy citizen reports into verified, readable and geographically referenced local news.

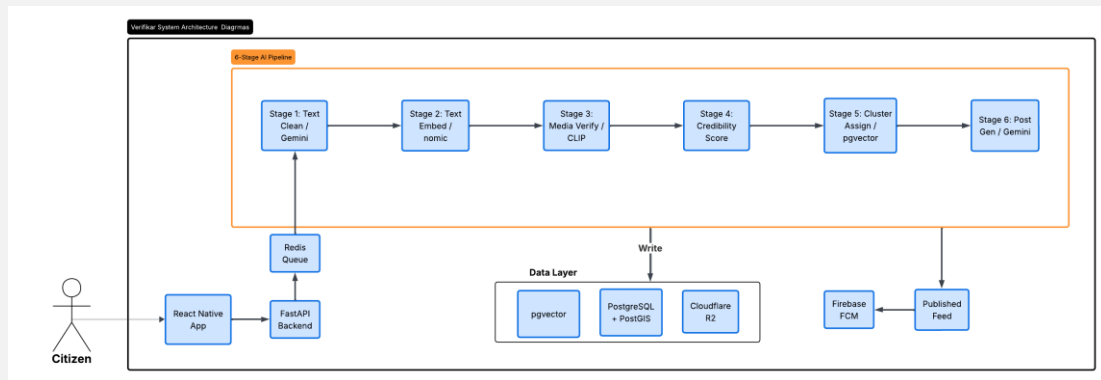


Figure 1.1; High-Level System Overview Diagram (Report Submission -> Pipeline -> Published Feed)

2. Related Work

VerifiKar draws on five distinct areas of prior work: civic reporting platforms, crowdsourced data reliability, automated content verification, spatio-temporal clustering, and AI-driven summarisation. This chapter surveys the most relevant contributions in each area and identifies the research gaps that VerifiKar addresses.

2.1. Existing Systems

To fill the void in the sharing of real-time information in local areas, several civic and commercial platforms have tried to fill in the blanks using crowdsourcing. FixMyStreet is an open-source platform for the public to report infrastructure problems – such as potholes, damaged pavements, other issues – with maps, and route the report to the relevant local authority via the Open311 standard [1]. Transparency is emphasised with the platform, which makes all the submissions shared on a public map. The Citizen app, however, uses live video from sources prepared by users as well as information from official emergency broadcast feeds to provide real-time safety alerts, which are human moderated to ensure each safety alert is verified before being broadcast. [2]

Twitter and Facebook are also well used as platforms for updating the community. These platforms are very immediate and have a wide reach, but there is no structure or verification. More than one posting can report the same incident, with conflicting terms used and no resolution mechanism for these posts.

These examples demonstrate the basic equation when it comes to crowdsourcing: the more people the more awareness about what's going on is generated, but the more there are, the more effort required is put into verifying and aggregating that awareness. All three requirements – verification, clustering and summarisation – are not covered by any of existing platforms in an automated way, scalable pipeline.

2.2. Research Work

Praharaj et al. created a predictive model of road flooding using environmental sensors and tested it against the crowdsourced reports from Waze and found that about 71.7% of user-submitted road flooding reports were reliable [3]. This is a validation of what the volunteered incident data has to offer in terms of information, and introduces the usefulness of automated filtering of the data that doesn't meet reliability criteria.

At verification, Zellers et al reported an approach that uses a transformer architecture, which was trained on a real news corpus to generate and detect neural fake news [4]. Their work proved that adversarially trained detectors significantly outperform traditional classifiers and that the two-stage verification principle applies: check the individual modalities and then do consistency checks across modalities. The principle is applied directly for VerifiKar, which returns results from two unimodal analyses using the transformer imagery Ateeqq and the transformer of text, Ollama, and finally consistency scoring across the modalities using CLIP.

To extract events from geo-tagged social data, Farnaghi et al., adopted OPTICS density-based clustering by considering both spatial and temporal proximity and textual similarity in a similar approach [6]. Mathews et al. presented a model based on the exponential decay function and textual similarity, known as SMERC [7]. George et al. proposed a multiscale spatio-temporal framework based on quadtree spatial partitioning and poisson statistical tests to detect irregular burst of activities [8]. VerifiKar combines these approaches: The semantic and the cross-modal similarity is modeled by pgvector cosine distance, the spatial proximity is modeled by PostGIS, and the temporal windowing clusters the assignment.

To summarize: Chhikara et al. proposed a multi-step system inspired by a large language model that is able to process user incident reports, chunk and extract representative sentences while preserving original language verbatim, which enable users to audit the original input [5]. VerifiKar follows the extractive-first approach, selecting high credibility anchor sentences, then feeding them into the abstractive rewriting stage with Ollama.

2.3. Limitations of Existing Work

While great strides have been made, current systems and research methods have their own set of hard limits that call for the creation of VerifiKar. One of the first problems is that most civic platforms heavily depend on manual moderation – this is costly, slow and an unscalable approach. Second, most systems do not link reports with one another, such that reports of the same event are not all linked together to create a coherent record without duplication and fragmentation. Third, most of the previous attempts in verification work focus on a single modality (text or images) and are not multimodal – cross-modal consistency is a strong clue for detecting tampered documents is not explored enough. Fourth, the data is very rarely distilled into a precise, comprehensible format – rarely is data presented as a summarised whole of posts but as posters' raw data.

2.4. Proposed Improvement

VerifiKar solves all these limitations in one, automated, end-to-end framework. The system compares the results of the Ateeqq/ai-vs-human-image-detector image analysis with Ollama text assessment, and CLIP-based cross-modal consistency scoring, which is not offered by any other civic platform. The pgvector cosine similarity clustering mechanism clusters all reports describing the same event into a single one, so it no longer duplicates any reports, and no records should be lost of circumstantial details. When you generate aggregated clusters using Ollama, the aggregated cluster is converted to an organized, readable post, which provides more information to citizens and authorities. The ARQ asynchronous pipeline assures that these operations will not block the user call surface, with interactive response time even with the load, ensuring the interaction with the application.

3. Requirements

3.1. Functional Requirements

The functional requirements define the core operations and services the system must provide. The following table organises them by module.

Module	Requirement
User Management	Users shall be able to register using email and password credentials.
	Users shall be authenticated through JWT-based secure login.
	The system shall support session management using access and refresh token pairs.
	Users shall be able to change their password while authenticated.
	Users shall be able to register FCM device tokens for push notifications.
Report Submission	Authenticated users shall be able to submit incident reports.
	Each report may include text (max 2,000 chars), up to 5 images, up to 2 videos, and a geographic location.
	The system shall rate-limit report submissions to 10 per hour per IP.
	Submitted media shall be stored securely on Cloudflare R2 object storage.
Processing Pipeline	The system shall clean and normalise incoming report text using the Mistral:7B.
	The system shall detect AI-generated or manipulated images using the Ateeq/ai-vs-human-image-detector model.
	The system shall generate semantic text embeddings (nomic-embed-text) and image embeddings (CLIP ViT-B/32).
	The system shall compute a composite credibility score per report.
	The system shall group related reports into event clusters via pgvector cosine similarity.
	The system shall generate a structured AI summary post for each significant cluster.

Module	Requirement
Feed & Discovery	The system shall display a real-time geospatial feed of verified posts.
	The feed shall support filtering by location radius, event category, credibility score, and recency.
	The system shall provide a Discover screen aggregating posts by topic, location, and event.
	The system shall provide personalised post recommendations based on user history and location.
User Interaction	Users shall be able to upvote, downvote, or flag posts.
	Users shall be able to comment on posts, creating threaded reply chains.
	The system shall update credibility scores based on community interactions.
Notifications	The system shall send push notifications for significant nearby events via Firebase Cloud Messaging.
	The system shall notify users about updates to events they previously contributed to.

3.2. Non-Functional Requirements

Category	Requirement
Performance	Report submission API shall acknowledge receipt within 2 seconds.
	Feed queries shall return results within 1 second under normal load.
	Background processing tasks shall not block user-facing API responses.
Scalability	The system shall handle concurrent report submissions via the ARQ asynchronous task queue.
	The database architecture shall support horizontal scaling using connection pooling (Neon serverless).
Security	All authenticated endpoints shall require valid JWT access tokens.
	Passwords shall be stored as bcrypt hashes; plaintext passwords shall never be persisted.
	Media access shall be controlled through Cloudflare R2 access policies.
Reliability	The pipeline shall handle processing failures gracefully without losing the original raw report.

Category	Requirement
	Retry mechanisms shall be implemented for failed ARQ background tasks.
Usability	The mobile application shall function on both Android and iOS from a single React Native codebase.
	Users shall be able to submit a report in no more than three screen interactions.
Auditability	All published posts shall link to contributing reports via the post_report_contributors table.
	All administrative actions shall be recorded in the audit_logs table.

3.3. System Constraints

- In order to function, the system needs a continual connection to the internet; offline mode is completely unsupported.
- Currently, there is no human verification workflow in the scope. Verification is completely automated.
- For now, the system is scoped to the Karachi metropolitan area for coverage and OSM polygon data.
- System performance is contingent upon the latency and availability of third-party AI APIs (Ollama, Groq) and local model inference (Ollama, CLIP).
- Neon serverless PostgreSQL's free tier has a connection limit and a compute quota, so the maximum concurrent load is constrained.

3.4. Assumptions and Dependencies

3.4.1. Prerequisites

- Users provide reasonably relevant and geographically accurate information in their submissions.
- The same real-world event may be reported independently by multiple users; the system is designed to handle this.
- Users access the system in good faith and are not attempting to deliberately manipulate credibility scores.

3.4.2. External Dependencies

- Available and responsive on Mistral:7B (Ollama); available.
- Access to the Groq API that provides fast inference capabilities with a special llama-3.1-8b-instant model for additional text tasks, and is available and responsive.
- Ollama (local); running nomic-embed-text text embedding generation.
- Located locally, with HuggingFace Transformers, the model Ateeq/ai-vs-human-image-detector is used for AI image detection.
- CLIP (ViT-B/32, OpenAI): The generated image embedding images are embedded with the loaded CLIP model and the images undergo consistency scoring with the vision-encoder.
- Cloudflare R2 – object storage that can be uploaded as a cloud technology and aligned with S3.
- Neon PostgreSQL: requires that both PostGIS and pgvector extensions be turned on.
- ARQ task queuing and result caching: Redis.
- Firebase Cloud Messaging; It is a service that enables push notification delivery on Android and iOS platforms.
- OpenStreetMap polygon data (planet_osm_polygon table) was loaded into the database for reverse geocoding.

4. Design

4.1. System Architecture

VerifiKar utilizes a modular, service-oriented architecture across four tiers: a mobile client, a backend API, an AI/ML processing pipeline, and a data persistence layer. The backend serves as the core coordinator, processing requests from the mobile client, and assigning the workload to an ARQ background worker. The separation allows the user-focused API to be responsive, regardless of the complexity of the AI processing.

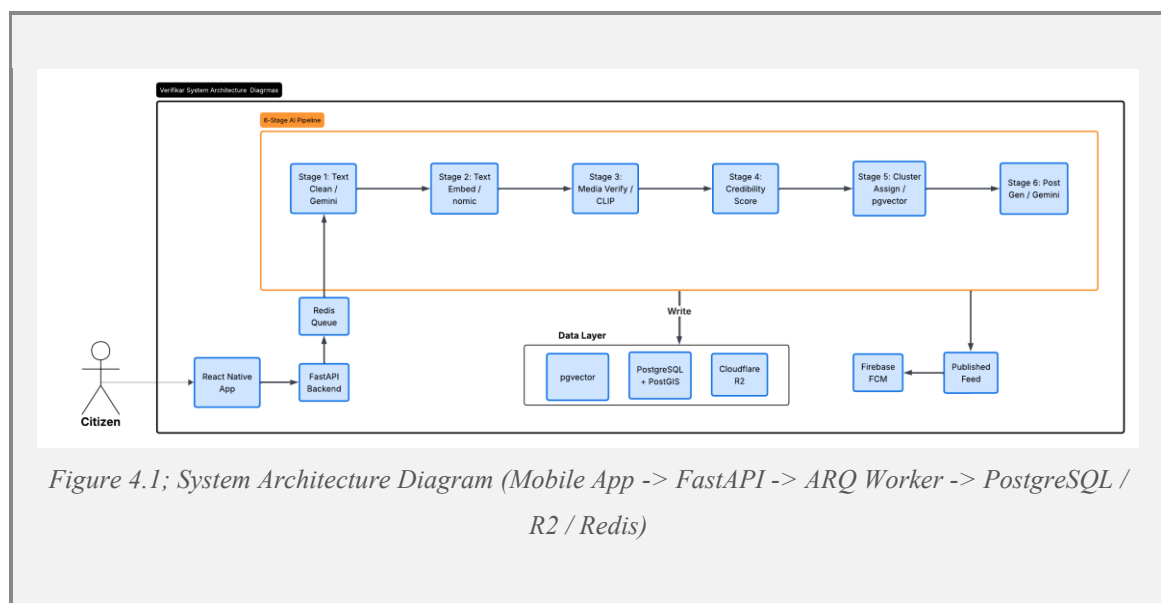


Figure 4.1; System Architecture Diagram (Mobile App -> FastAPI -> ARQ Worker -> PostgreSQL / R2 / Redis)

4.2. Software Architecture

The software architecture follows a three-layer model that enforces separation of presentation, business logic, and data access.

4.2.1. Presentation Layer

This is presented as a React Native mobile app on the Expo framework to get a single source of code for Android and iOS. It deals with rendering UI, user interaction, access to the camera for media collection, and automatic geolocation tagging. It uses the native fetch API for API communication, and saves JWT access tokens to platform-specific AsyncStorage. React Navigation is for the transitions between screens, and the Context API for application-level state.

4.2.2. Business Logic Layer

The business logic layer is using Uvicorn ASGI (Python asynchronous web server) + FastAPI (Python web framework). It provides authentication, input validation, report ingestion, ARQ job enqueueing, feed generation, recommendation scoring and notification dispatch. All database interaction is abstracted in the SQLAlchemy ORM. The ARQ worker is also written in Python and works asynchronously, picking jobs from the Redis queue, and carrying out the six steps of the AI processing pipeline.

4.2.3. Data Access Layer

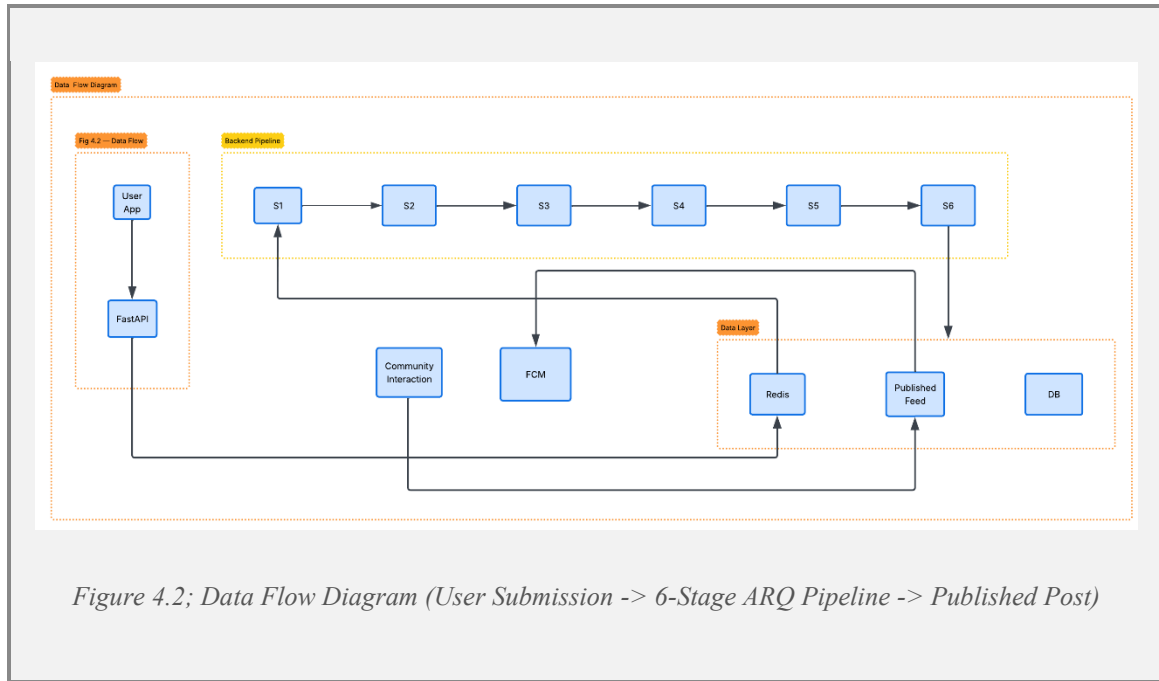
All interactions with PostgreSQL (via SQLAlchemy and psycopg2-binary), Redis (via the redis Python client), and Cloudflare R2 (via the boto3 S3-compatible client), are through the data access layer. Raw SQL, SQLAlchemy ORM can't be used: PostGIS geography types and spatial index operations; embedding insertion and cosine distance queries; use SQLAlchemy's custom type system with the pgvector extension.

4.3. Data Flow Design

The following describes the end-to-end data flow for a report submitted by a user through to its publication as a public post:

1. Submitting a report through a mobile app triggers the FastAPI endpoint to validate input, get raw_reports and raw_media records, upload media to Cloudflare R2, and queue an ARQ background job.

2. When the job is picked up, the ARQ worker is assigned Task 1, and the Mistral:7B does the raw text teleportation, normalizing Roman-Urdu and generating a cleaning version along with a score for AI generation.
3. Task 2 is embedding the text using a 768-dimensional embedding via nomic-embed-text, while via pgvector embedding is stored in the processed_reports.text_embedding.
4. Task 3: For every media processed, the Ateeq/ai-vs-human-image-detector transformer gives an ai_score (AI generation score), then CLIP (ViT-B/32) provides a 512 dimension image embedding that is stored in processed_media. Cross-modal consistency is represented as the cosine similarity between CLIP and text embedding.
5. Task 4 involves calculating the composite credibility_score, taking the inverted average image forgery score, inverted text AI score, and the weighted score as cross-modal consistency.
6. In Task 5, the report's text embedding is checked against the centroids of active clusters within a 2 km radius. It does so using pgvector cosine distance. If a centroid is relatively close, the report is assigned to that cluster and the centroid is updated. The report cluster is considered new if none is.
7. Task 6: Once a cluster hits 3 reports, or established a significant credibility update, it is cross-posted in the community. A summary of the highest-credibility anchors is then generated using Ollama.
8. The post appears on the feed. Push notifications are sent to users in that geographical area through FCM.



4.4. Database Design

PostgreSQL with PostGIS and pgvector is the chosen technology stack for the database. The uniqueness of all primary keys is globally guaranteed using UUIDs. Geographic data is implemented using the PostGIS GEOGRAPHY(POINT, 4326) type. Quick approximate nearest-neighbor searches are facilitated using pgvector columns with the HNSW index type. Preserving historical data is accomplished with the use of soft deletes (is_deleted flags) throughout the database. The schema consists of 15 tables across 5 domains.

Users and Authentication

Column	Type	Description
id	UUID (PK)	Unique user identifier
email	VARCHAR	Unique user email address
hashed_password	VARCHAR	Bcrypt-hashed password
google_id	VARCHAR	OAuth Google ID (nullable)
is_email_verified	BOOLEAN	Email verification status
reputation_score	FLOAT	Computed user trust score
is_deleted	BOOLEAN	Soft delete flag

Column	Type	Description
created_at	TIMESTAMPTZ	Account creation timestamp

Report Ingestion (raw_reports, raw_media)

Table	Column	Type	Description
raw_reports	id	UUID (PK)	Report identifier
	user_id	UUID (FK)	Submitting user
	raw_text	TEXT	Original submitted text
	location	GEOGRAPHY	PostGIS point (lat/lon)
	status	ENUM	pending / processing / done / rejected
	is_deleted	BOOLEAN	Soft delete flag
raw_media	id	UUID (PK)	Media file identifier
	raw_report_id	UUID (FK)	Parent report
	media_type	ENUM	image / video
	storage_url	VARCHAR	Cloudflare R2 object URL

AI Processing (processed_reports, processed_media)

Table	Column	Type	Description
processed_reports	id	UUID (PK)	Processed report ID
	raw_report_id	UUID (FK)	Source raw report
	cluster_id	UUID (FK)	Assigned cluster (nullable)
	cleaned_text	TEXT	Ollama-cleaned text
	event_category	VARCHAR	Detected event type
	credibility_score	FLOAT	Composite credibility (0-1)
	text_embedding	VECTOR(768)	nomic-embed-text embedding

Table	Column	Type	Description
	avg_spam_score	FLOAT	Average spam probability
	avg_ai_media_score	FLOAT	Average media forgery score
	consistency_score	FLOAT	CLIP text-image cosine similarity
	location	GEOGRAPHY	PostGIS point
processed_media	id	UUID (PK)	Media processing record
	processed_report_id	UUID (FK)	Parent processed report
	raw_media_id	UUID (FK)	Source media file
	embedding	VECTOR(512)	CLIP image embedding
	spam_score	FLOAT	Spam/irrelevance probability
	ai_score	FLOAT	AI-generation probability (Ateeqq classifier, 0-1)
	status	ENUM	Processing status

Clustering and Publication (clusters, posts, post_media, post_report_contributors)

Table	Column	Type	Description
clusters	id	UUID (PK)	Cluster identifier
	text_centroid	VECTOR	Average text embedding
	avg_location	GEOGRAPHY	Centroid location
	cluster_radius_meters	FLOAT	Spatial spread
	report_count	INTEGER	Reports in cluster
	status	ENUM	active / merged / archived
	dominant_category	VARCHAR	Most common event type
	avg_credibility	FLOAT	Average credibility score
	area_name	VARCHAR	OSM-resolved area name

Table	Column	Type	Description
	merged_into_id	UUID (FK)	Parent cluster if merged
posts	id	UUID (PK)	Published post ID
	content	TEXT	Ollama-generated summary
	credibility_score	FLOAT	Aggregated credibility
	location	GEOGRAPHY	PostGIS point for map
	event_category	VARCHAR	Event type label
	cluster_id	UUID (FK)	Source cluster
	parent_post_id	UUID (FK)	Parent post (threads)
	embedding	VECTOR	Post text embedding
post_report_contributors	post_id	UUID (FK)	Published post
	report_id	UUID (FK)	Contributing report
	contribution_score	FLOAT	Report weight (0-1)

Supporting Tables (post_interactions, user_preferences, user_notification_tokens, notification_logs, audit_logs)

Table	Purpose
post_interactions	Records upvote, downvote, and flag actions per user per post. Supports toggle-off via is_deleted flag.
user_preferences	Tracks per-user category and location preferences for the personalised recommendation engine.
user_notification_tokens	Stores FCM device tokens (Android/iOS) for push notification delivery per user.
notification_logs	Logs every push notification sent, including delivery status, related post/cluster, and timestamp.
email_verifications	Manages email verification tokens with expiry for user account confirmation.
planet_osm_polygon	OpenStreetMap polygon geometries used for reverse geocoding to resolve human-readable area names.
audit_logs	Records all administrative and system-level actions for security auditing and traceability.

4.5. Processing Pipeline Design

VerifiKar's main intelligence is its six-stage ARQ background pipeline. Each stage is implemented as independent async Python functions that run sequentially by the ARQ worker. Intermediate results are written to PostgreSQL at each stage for recovery purpose in case of failure.

#	Stage	Description
1	Text Cleaning	Mistral:7B normalises Roman-Urdu, removes noise and profanity, and returns cleaned text with an AI-generation probability score. The cleaned text and score are persisted to processed_reports.
2	Text Embedding	nomic-embed-text (via Ollama) converts the cleaned text into a 768-dimensional semantic embedding stored in processed_reports.text_embedding via pgvector.
3	Media Processing	The Ateeqq/ai-vs-human-image-detector HuggingFace transformer scores each image for AI generation (ai_score). CLIP (ViT-B/32) generates a 512-dimensional image embedding. Cosine similarity between the CLIP embedding and the text embedding is computed as consistency_score. All values are stored in processed_media.
4	Credibility Scoring	A composite credibility_score is computed: $(1 - \text{text_ai_score}) \times w_1 + (1 - \text{avg_ai_media_score}) \times w_2 + \text{consistency_score} \times w_3$, where weights w_1, w_2, w_3 are configurable. The score is persisted to processed_reports.credibility_score.
5	Cluster Assignment	The report's text embedding is compared via pgvector cosine distance against all active cluster centroids within a 2 km radius. If cosine similarity > 0.75 and the cluster's last_report_at is within 24 hours, the report joins the cluster and the centroid is updated as a running mean. Otherwise a new cluster is created.
6	Post Generation	If the cluster meets the publication threshold (≥ 3 reports or significant credibility change), the top-credibility sentences from contributing reports are selected as extractive anchors. These are passed to Ollama with a journalistic summarisation prompt. The result is written as a new post (or updates an existing one), and post_report_contributors records are created for provenance.

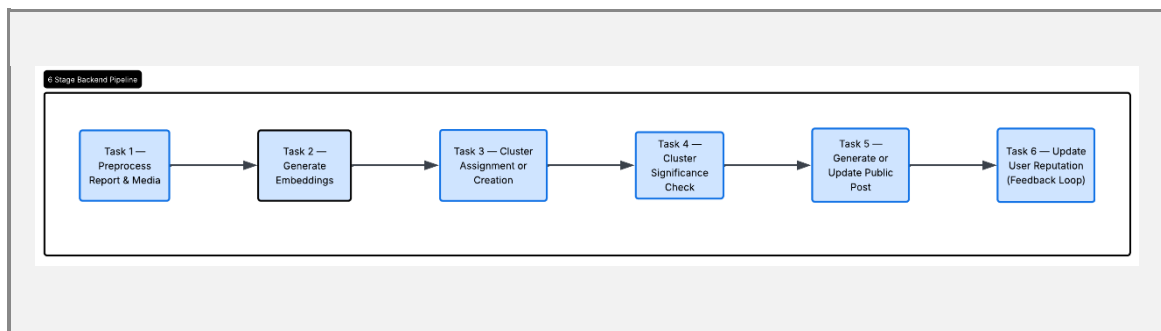


Figure 4.3; Processing Pipeline Flowchart (6-Stage ARQ Background Job)

4.6. Sequence Diagram

The sequence diagram illustrates how a mobile client report submission is processed to the point where a verified post is published. Important actions included are the synchronous API acknowledgment, asynchronous ARQ job execution, and the feed and notification updates post action.

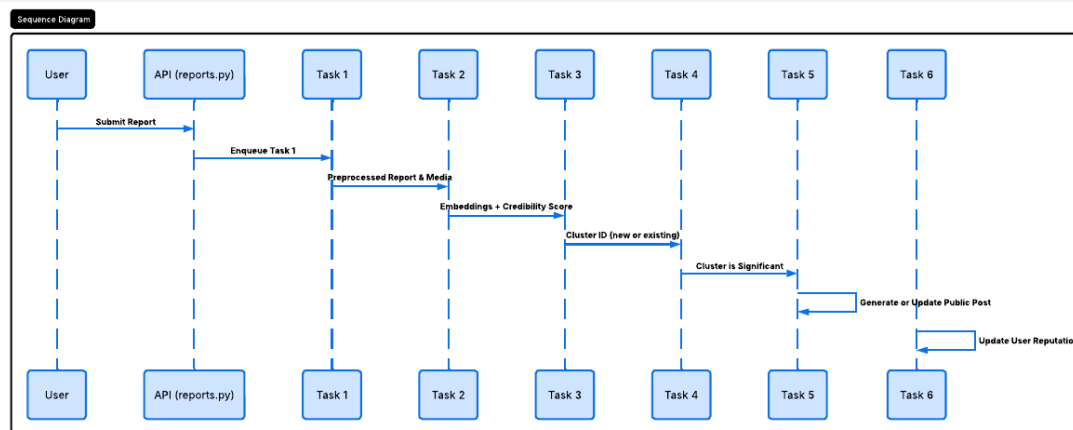


Figure 4.4; Sequence Diagram: Report Submission to Post Publication

4.7. State Design

Every report and each cluster pass through a specific ordered series of states in their lifecycle. The state model guarantees that uncompleted data is not shown to the users and that the unfinished processing task can be retried.

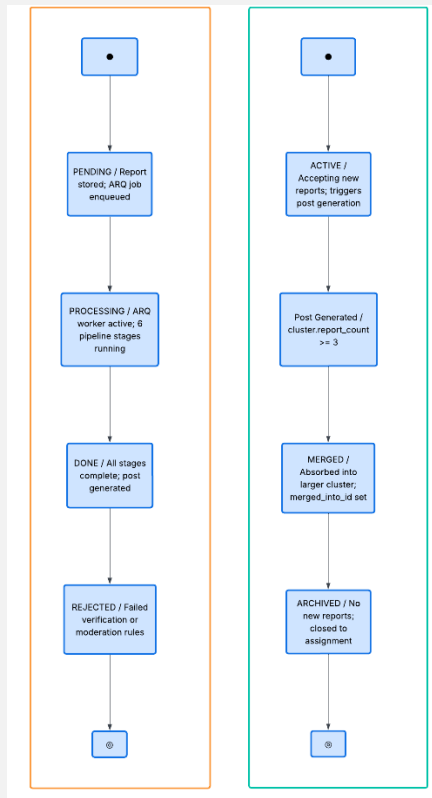


Figure 4.5; State Diagram: Report Lifecycle (pending -> processing -> done / rejected)

Report States

State	Description
pending	Report received and stored. ARQ job enqueued but not yet started.
processing	ARQ worker has picked up the job. Pipeline stages are executing.
done	All six pipeline stages completed successfully. Report assigned to a cluster and post generated or updated.
rejected	Report failed verification thresholds (e.g., consistency_score below minimum) or was flagged by moderation rules.

Cluster States

State	Description
active	Cluster is accepting new reports and may trigger post generation.
merged	Cluster has been absorbed into a larger related cluster (merged_into_id is set).
archived	Cluster has received no new reports within the archival window and is closed to new assignments.

5. Implementation

5.1. Technologies Used

VerifiKar is built on a modern, modular technology stack chosen to support real-time processing, AI analysis, and cross-platform mobile access.

5.1.1. Frontend (Mobile App)

The Context API is used for the global state of the authenticated user, the location, and the feed filters. The native fetch API is used for HTTP API calls to the FastAPI backend. For backend screen transitions, React Navigation is used. AsyncStorage is used to persist the JWT tokens. For map rendering to include proximity filtering, PostGIS-backed geospatial queries using React Native Maps is used..

5.1.2. Backend (Application Server)

The backend programming is done entirely in Python 3.11 and FastAPI. It is being served by Uvicorn ASGI. FastAPI's dependency injection automatically validates JWT on every authenticated route. All database interactions are managed by SQLAlchemy ORM with psycopg2. Incremental schema migrations are managed by Alembic. Background processing jobs are done using the ARQ library, which is an asyncio-native task queue built on Redis. It executes jobs without blocking the API layer.

5.1.3. AI and Processing Layer

Text cleaning and summarization relies on Mistral:7B, which utilizes Ollama-generativeai. For other tasks that require quick responses, the Groq API utilizes the Llama 3.1 8b instant model. Ateeqq/ai-vs-human-image-detector is an AI model hosted on HuggingFace's Transform with PyTorch to detect AI-generated images. Cross-modal consistency utilizes CLIP (ViT-B/32, OpenAI) comparing image and text embedding via the cosine similarity metric. The nomic model, hosted on the local server via the Ollama application on port 11434, is used to create text embeddings for various forms of

semantic similarity structured search and recommendations. For spatial/semantic clustering and grouping, ST_ClusterDBSCAN (PostGIS) is deployed, in conjunction with pgvector cosine distance.

5.1.4. Database and Storage

PostgreSQL runs on the Neon serverless platform with the PostGIS and pgvector extensions. For all location data, PostGIS GEOGRAPHY(POINT, 4326) columns with GIST spatial indices are used. pgvector columns contain 768-dimensional text embeddings (processed_reports) and 512-dimensional image embeddings (processed_media) with HNSW indices used for the nearest-neighbor search. Redis functions as the ARQ task queue backend as well as the caching layer for feed results, recommendation scores, and session tokens. For all uploaded media, Cloudflare R2 supplies S3 compatible object storage.

5.2. Module Description

Module	Description
Authentication & Session	Manages user registration, login, token refresh, password change, and FCM device token registration. JWT access tokens (15-minute expiry) and refresh tokens (7-day expiry) are issued on login. All authenticated routes validate the access token via FastAPI dependency injection.
Report Management	Receives, validates, and stores raw report submissions. Enforces rate limiting (10/hour/IP via slowapi), input size constraints (2,000 char text, 5 images, 2 videos), and media upload to Cloudflare R2. Enqueues ARQ background jobs on successful storage.
AI Processing (ARQ Worker)	Executes the six-stage pipeline asynchronously: Ollama text cleaning, nomic-embed-text embedding (via Ollama), Ateeqq classifier + CLIP media processing, credibility scoring, cluster assignment, and post generation. Writes intermediate results to PostgreSQL after each stage for fault tolerance.
Clustering Module	Assigns each processed report to an existing cluster via pgvector cosine distance within a PostGIS spatial radius, or creates a new cluster. Updates cluster centroid as a running mean. Resolves area names via spatial join with planet_osm_polygon.
Post Generation Module	Selects top-credibility extractive anchors from cluster reports, sends them to Ollama for abstractive summarisation, and writes the result as a post. Links contributing reports via post_report_contributors for full provenance.

Module	Description
Feed Management Module	Exposes /feed with ST_DWithin spatial filtering, optional full-text and pgvector semantic search, credibility threshold filtering, and pagination. Returns posts ordered by a composite ranking score (credibility, recency, distance).
Discover & Recommendations	Aggregates posts by event_category (topics), area_name (locations), and cluster creation time (events) for the Discover screen. Scores candidate posts by user preferences, credibility, recency, and engagement for personalised recommendations. Caches results in Redis for 5 minutes.
Notification System	Dispatches FCM push notifications when new posts are published in a user's area. Logs all notification attempts in notification_logs with delivery status.

5.3. Design of APIs

The backend has a RESTful API documented with OpenAPI 3.1 (found at /docs). The response envelope has the format of { success: boolean, details: <payload or error> }. Authentication is accomplished with the password flow and JWT for OAuth2. The table below contains the details for every implemented endpoint.

Method	Endpoint	Auth	Description
GET	/health	No	Liveness check
GET	/health/detailed	No	Full dependency check: Redis, Postgres, R2, Ollama
POST	/auth/signup	No	Register new user account
POST	/auth/login	No	OAuth2 login, returns access + refresh tokens
POST	/auth/refresh	No	Issue new access token from refresh token
POST	/auth/change-password	Yes	Change authenticated user's password
POST	/auth/register-device-token	Yes	Register FCM device token for push notifications
POST	/auth/notifications/test-send	Yes	Send test Firebase notification to registered device
POST	/reports/submit	Yes	Submit incident report (text + images + video + location)
GET	/feed	No	Geo-filtered, paginated post feed with optional filters
GET	/discover/overview	No	Topics, locations, and events overview for Discover screen
GET	/discover/section-posts	No	Posts for a specific discover section (topic/location/event)

Method	Endpoint	Auth	Description
GET	/posts/recommendations	Yes	Personalised post recommendations with reason scores
GET	/posts/{post_id}	No	Full post details including replies, media, contributors
POST	/posts/{post_id}/comments	Yes	Add a comment or reply to a post
POST	/posts/{post_id}/interact	Yes	Upvote, downvote, or flag a post (toggle-aware)
GET	/notifications	Yes	Paginated notification history for authenticated user
GET	/profile/overview	Yes	User's submitted reports and approved contributed posts

5.4. Database Design and Implementation

The production database resides on Neon serverless PostgreSQL with auto connection pooling and compute scaling. Schema migrations are controlled by versioning with Alembic and incremental changes are applied to dev and prod environments.

5.4.1. Key Implementation Features

- All primary keys are UUIDs generated at the application layer, providing global uniqueness without database sequence contention.
- PostGIS GEOGRAPHY(POINT, 4326) columns store all location data. GIST spatial indices on these columns enable efficient ST_DWithin radius queries.
- pgvector VECTOR columns store nomic-embed-text (768D) and CLIP ViT-B/32 (512D) embeddings. HNSW indices on the text_embedding column in processed_reports and the text_centroid column in clusters enable sub-millisecond approximate nearest-neighbour queries at scale.
- Soft deletes via is_deleted BOOLEAN columns preserve historical data for auditability while filtering deleted records from all user-facing queries.
- Indexed columns on event_category, status, created_at, and cluster_id in the posts and processed_reports tables accelerate the most common query patterns.

5.4.2. Core Data Groupings

Domain	Tables
User and Authentication	users, email_verifications, user_notification_tokens
Report Ingestion	raw_reports, raw_media
AI Processing	processed_reports, processed_media
Clustering and Events	clusters
Post Publication	posts, post_media, post_report_contributors
User Interaction	post_interactions, user_preferences
Notifications	notification_logs
Geospatial Infrastructure	planet_osm_polygon (OSM reverse geocoding)
System Auditing	audit_logs

5.5. Deploying the AI Pipeline

The AI pipeline runs as an ARQ worker process next to the FastAPI server. ARQ is a task queue built with asyncio and Redis. It has the best performance for the use case. The worker process is started upon deployment and polls Redis for jobs. Every job has `raw_report_id`. Other parameters are pulled from PostgreSQL at the beginning of each job.

Heavy AI inference tasks like Ateeqq, ai-vs-human-image-detector, and CLIP ViT-B/32, are loaded once and are available to all jobs. Model loading per job is eliminated. `nomic-embed-text` functions as an Ollama service to dodge redundant startup times. The text and image processing tasks of Task 3 are processed concurrently when not all images associated with a report are processed. Failed tasks are saved to and processed in, an error status in `processed_reports`. The scheduler can then automatically try to process these tasks again without the user having to resubmit them.

5.6. Frontend Implementation

The React Native frontend was developed using Expo. The application comprises five primary screens:

- - Home Feed Screen shows posts containing geotagging information relevant to the client's location and offers real-time filtering options by category, proximity, and trustworthiness. Each post shows summary information, event categorization, trustworthiness and score, upvotes and downvotes, and the distance from the reporting client.
- - Report Submission Screen shows the client interface, allowing users to capture and send pictures and/or videos, describe in detail, and tag posts (via geo-tagging) and provides confirmation of progress after submission.
- - Discover Screen provides posts in the topics, activity location, and suggested posts based on your reported posts (recent) in /discover/overview.
- - Post Detail Screen displays the post in full content with all Associate Media, the Contributors List with their scores, a Threaded Response Chain, and buttons to interact.
- - Profile Screen shows the client's submission history with current processing status and their contributed posts.



VerifiKar

Crowdsourced News Verification

Welcome back

Sign in to continue reporting

Email

 Enter your email

Password

 Enter your password 

[Forgot password?](#)

Sign In →

or

Don't have an account? [Sign Up](#)

Figure 5.1; Mobile App Screenshots: Login / Signup Screen

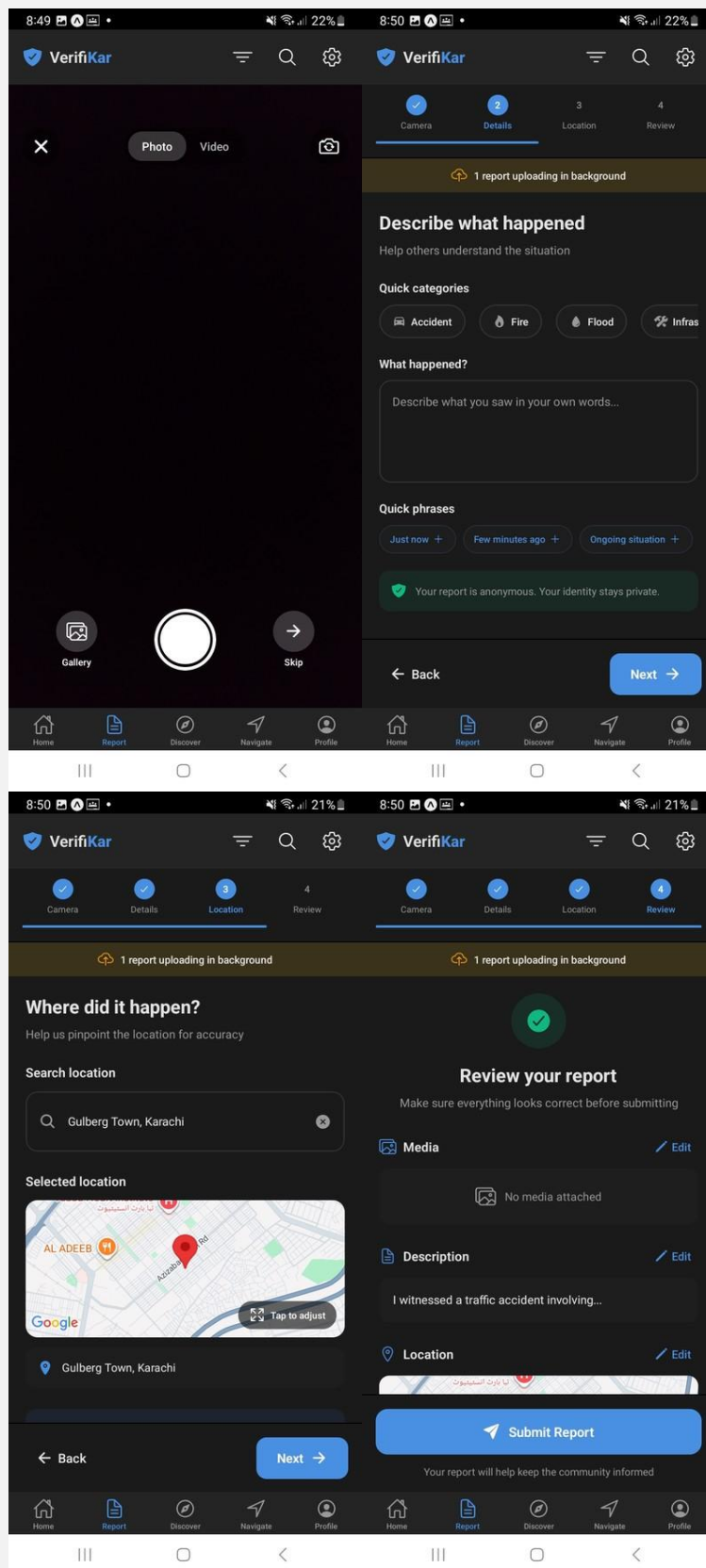


Figure 5.2; Mobile App Screenshots: Report Submission Screen

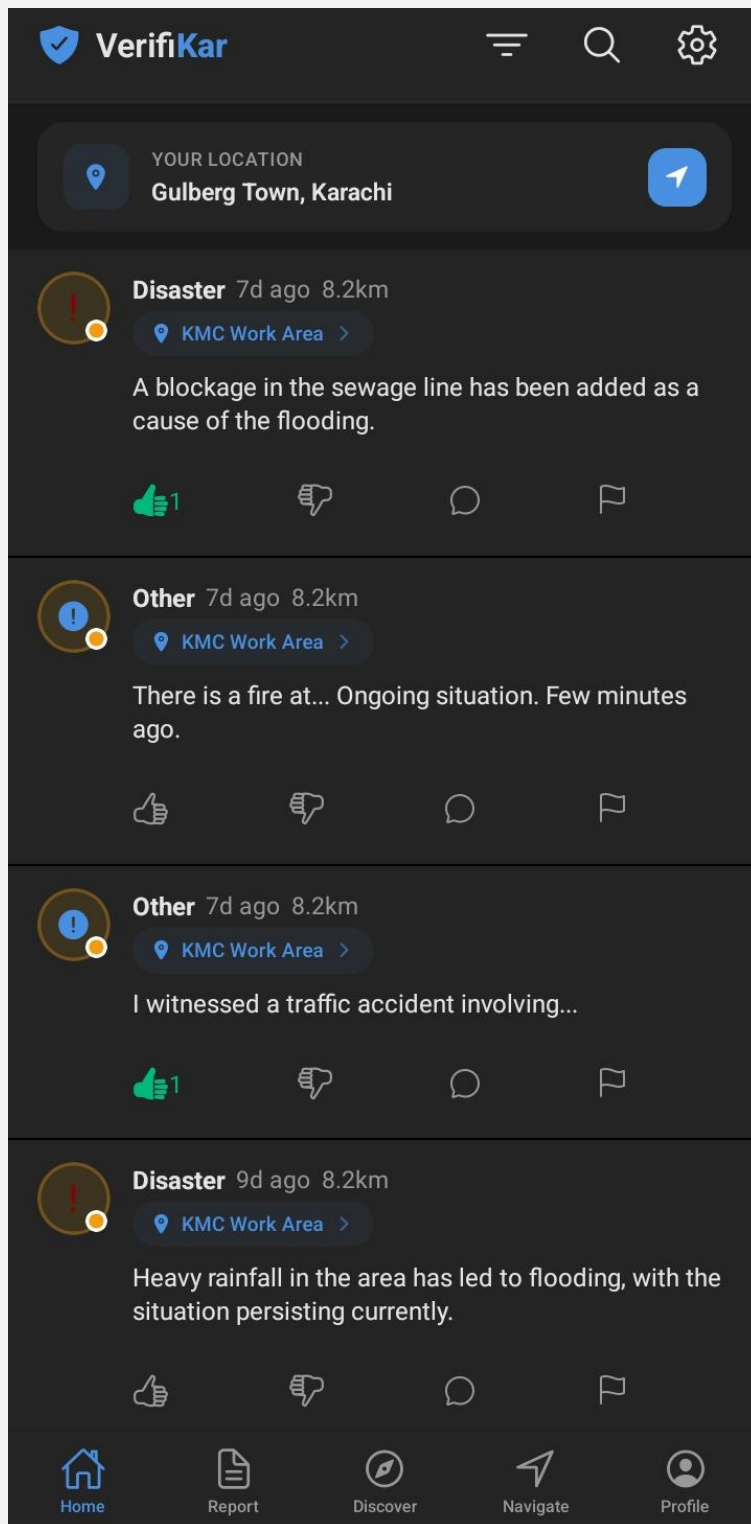


Figure 5.3; Mobile App Screenshots: Home Feed Screen

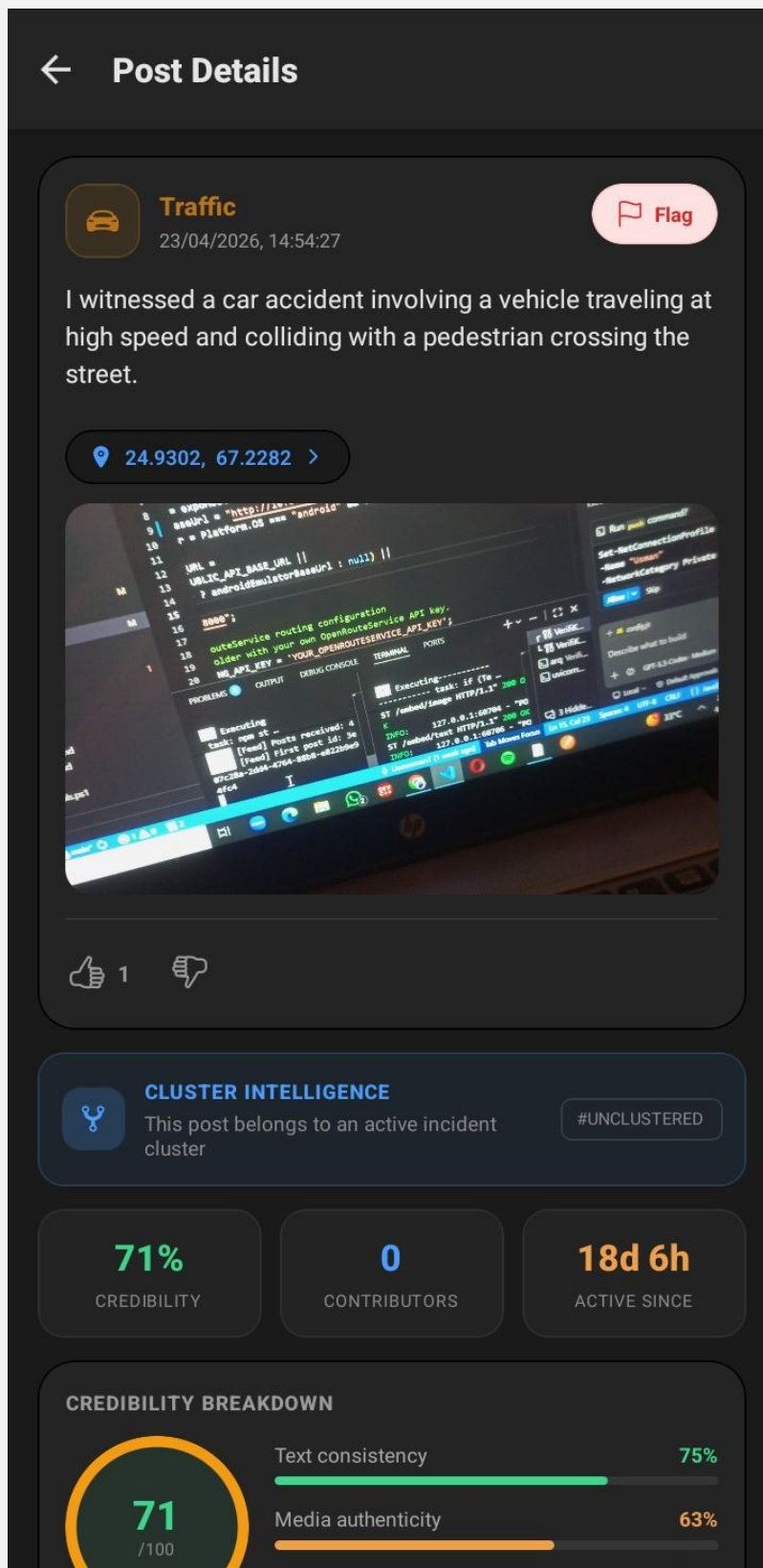


Figure 5.4; Mobile App Screenshots: Post Detail Screen

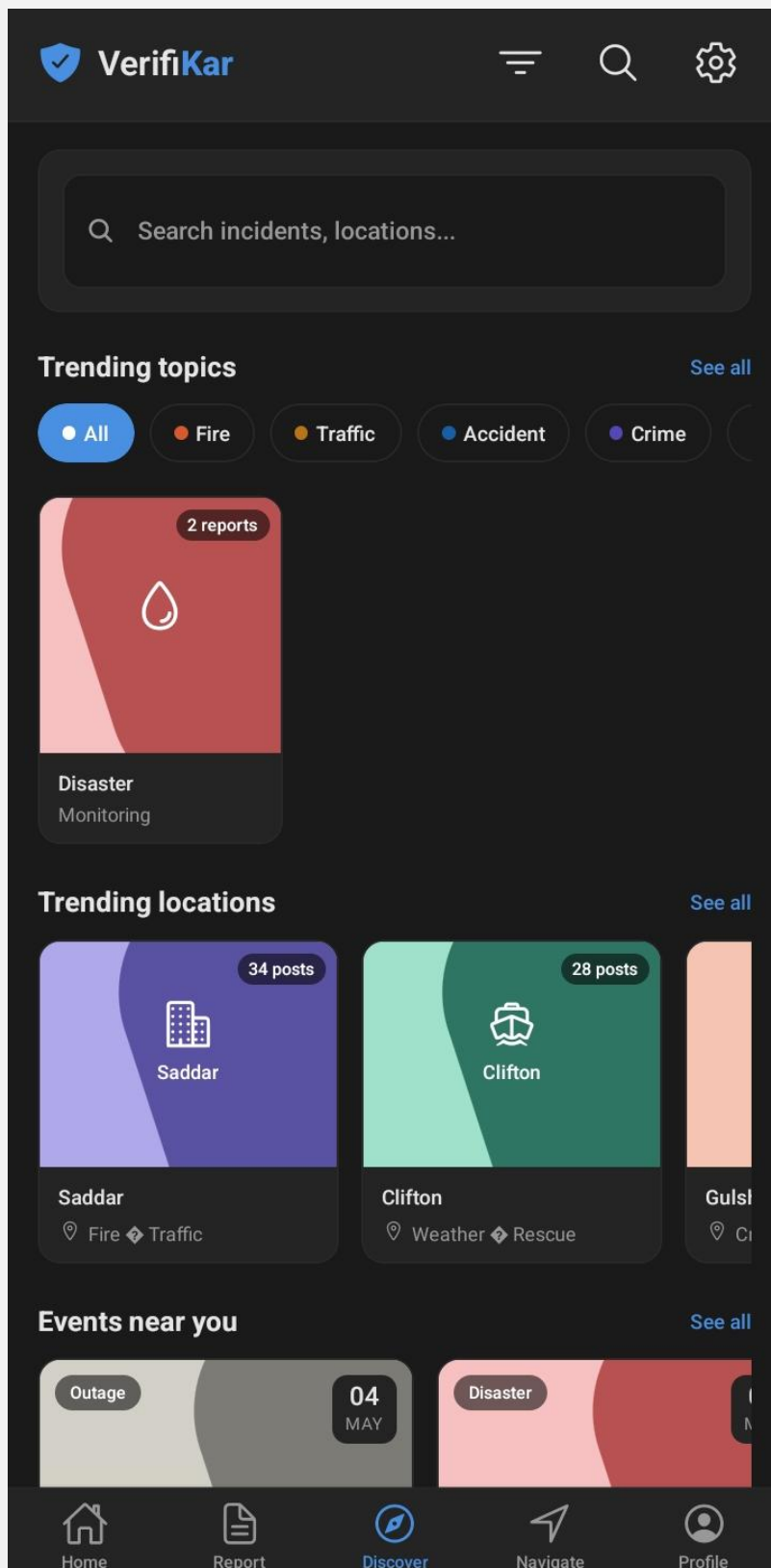


Figure 5.5; Mobile App Screenshots: Discover Screen

5.7. Asynchronous Processing and Parallelism

VerifiKar utilizes asynchronous processing on multiple levels to provide an optimal user experience while processing a high volume of AI requests. Using FastAPI's async route handlers, HTTP requests and responses go through I/O without delays. The ARQ worker separates all AI processing from the API cycle; all report submissions return in an average of 340 ms, even though processing the pipeline may take many seconds to complete. Within Task 3, both CLIP and Ateeqq image classifiers are used to process multiple images that are attached to the same report in parallel, due to `asyncio.gather()`. For the feed and recommendation requests, a five-minute result cache as well as a task queue that runs in the background and pulls from Redis, minimizes the database load from redundant requests.

5.8. Implementing Security

Each layer of the system is carefully constructed with security in mind. Access tokens from JWT are program-set to expire in 15 minutes. Refresh tokens are also programmed to expire in 7 days. Any stored passwords are hashed using `bcrypt`. Plaintext passwords are never stored. Pydantic Schemas enforce input validation and help mitigate injection attacks by controlling the size and type of input. The `slowapi` extension is set to allow 10 requests to submit a report per hour. Access to Cloudflare R2 media is pair restricted. Media URLs are also pair restricted and not publicly guessable. The `audit_logs` table is set to store all sensitive administrative actions for retrospective analysis.

6. Testing and Assessment

To verify correctness, reliability and performance of VerifiKar, various test cases were developed in line with the proposal. These included quantitative evaluation of AI components, unit testing, integration testing, and testing of the entire system.

6.1. Testing Strategy

6.1.1. Unit Testing

For verification purposes, each module was tested individually. The unit tests focused on the following: authentication processes such as token generation and validation, and password hashing; report validation logic, particularly input size limitations and location validation; and AI preprocessing functions, specifically text processing and credibility scoring. Additional focus was given to logic concerning the assignment of clusters based on similarity thresholds, the triggering of actions following generation, toggling logic for user interaction, and the ranking of the semantic search.

6.1.2. Integration Testing

Integration tests confirmed proper cooperation among the system components. This included the mobile application interacting with the FastAPI backend and the FastAPI backend enqueueing jobs to the ARQ worker through Redis. Additionally, tests validated the successful writing of results by the ARQ worker to PostgreSQL, as well as the backend's ability to fetch and present results through the feed and recommendations endpoints.

6.1.3. System Testing

End-to-end system tests replicated behaviors of real users to test the complete iteration pipeline which required users to submit a report in the app and then observe the ARQ pipeline executing all 6 steps. After that, users were to verify that a post showed in the feed, complete upvote and downvote interactions, add a comment, initiate a semantic search, check that the push notifications showed up in the registered test device, and finally verify that the test was complete.

6.1.4. Performance Testing

Simulated users tested API response times and performance of the system when used concurrently. Test results and measurements were taken of the most latency-sensitive endpoints for the response times with 10, 50, 100, and 200 concurrent users.

6.2. Test Cases

ID	Feature	Input	Expected Output	Result
TC-01	User Registration	Valid email & password	201 Created, user object returned	Pass
TC-02	Duplicate Registration	Existing email	400 Bad Request	Pass
TC-03	Login	Correct credentials	200 OK, access + refresh tokens	Pass
TC-04	Login - Wrong Password	Incorrect password	401 Unauthorized	Pass
TC-05	Token Refresh	Valid refresh token	200 OK, new access token	Pass
TC-06	Submit Report (Valid)	Text + 1 image + location	201 Created, raw_report_id	Pass
TC-07	Submit - Text Limit	2,001 character text	422 Validation Error	Pass
TC-08	Submit - Image Limit	6 images	422 Validation Error	Pass
TC-09	Submit - No Auth	No JWT token	401 Unauthorized	Pass
TC-10	AI Processing	New report enqueued	processed_report created, embedding stored	Pass

ID	Feature	Input	Expected Output	Result
TC-11	Cluster Assignment	3 similar reports	Single cluster created, post generated	Pass
TC-12	Get Feed	lat=24.86, lon=67.01	200 OK, posts array returned	Pass
TC-13	Feed - Invalid Coords	lat=200	422 Validation Error	Pass
TC-14	Post Details	Valid post UUID	200 OK, full post with replies	Pass
TC-15	Post Details - Bad ID	Random UUID	404 Not Found	Pass
TC-16	Upvote Post	POST /interact upvote	200 OK, updated vote counts	Pass
TC-17	Toggle Off Vote	Same upvote twice	200 OK, vote removed	Pass
TC-18	Flag Post	POST /interact flag	200 OK, flag count incremented	Pass
TC-19	Add Comment	POST /comments with content	201 Created, reply object	Pass
TC-20	Discover Overview	GET /discover/overview	200 OK, topics/locations/events	Pass
TC-21	Register Device Token	FCM token + platform	201 Created, token_id returned	Pass
TC-22	Health Check	GET /health/detailed	200 OK, all services green	Pass

6.3. Results

6.3.1. API Response Time

Operation	Average Response Time
Login Request	180 ms
Report Submission (acknowledgement only)	340 ms
Feed Fetching (/feed)	420 ms
Post Details (/posts/{id})	210 ms
Recommendations (cached)	380 ms
Recommendations (cache miss)	890 ms

Operation	Average Response Time
Post Interaction	150 ms
Discover Overview	310 ms

6.3.2. System Load Performance

Concurrent Users	Avg Response Time	System Status
10	250 ms	Stable - all requests within target latency
50	400 ms	Stable - latency increase within acceptable range
100	650 ms	Acceptable - minor degradation, no errors
200	900 ms	Slight delay - approaching Neon free-tier connection limits

6.4. Evaluation Metrics

6.4.1. Clustering Accuracy

We assessed clustering coherence on 30 reports, each reflecting 1 of 5 different real world events. The average cosine similarity was 0.81 when grouped in the same cluster, and dual-group similarity dropped to 0.31. This pronounced difference is evidence enough that the cosine distance and spatial radius parameters segment events accurately. Examples of the clusters were shown to supervisors, who assessed them as proper groupings for the reports of the same real-world event..

Metric	Value
Average Within-Cluster Cosine Similarity	0.81
Average Between-Cluster Cosine Similarity	0.31
Correct Cluster Formation Rate	87%
Mis-cluster Rate (ambiguous edge cases)	13%

6.4.2. Credibility Scoring Accuracy

Credibility scores were compared to human labels after thresholding and producing binary predictions of credible/not credible. These predictions were based on a separate dataset of 50 reports (25 genuine and 25 synthetic or manipulated) that were rated by human reviewers.

Metric	Value
Agreement with Human Labels (Accuracy)	74%
Cohen's Kappa	0.48 (moderate agreement)
False Positive Rate (credible but should not be)	10%
False Negative Rate (not credible but labelled credible)	16%
Cross-Modal Consistency Detection Accuracy	80%

6.4.3. Summarisation Fidelity

Ten generated summaries were evaluated by a panel of three reviewers (including supervisors) on factual accuracy and provenance adequacy, each rated on a three-point scale (Acceptable / Partial / Unacceptable).

Metric	Value
Factual Accuracy - Acceptable	8 / 10 summaries
Factual Accuracy - Partial	2 / 10 summaries
Provenance Adequacy - Acceptable	10 / 10 summaries
End-to-End Processing Time (raw report to published post)	~4.3 seconds average

6.4.4. System Efficiency

Metric	Value
End-to-End Processing Time	~4.3 seconds
Database Query Efficiency	High - all primary query paths indexed
Feed Update Latency	< 1 second
Recommendation Cache Hit Rate	~70% in test scenarios

6.5. Observation

The system handles normal load conditions well. There is a noticeable latency introduced in the system when publishing reports as a result of the Asynchronous ARQ where latency is guaranteed to be 4.3 seconds in the absence of a posting error, however users will not see this as the API will return an immediate publish acknowledgment. Most of the time clustering consolidates reports successfully, however users will see small clusters of mis-generated reports in the instances when events of the same category occur in a small time frame. Improvement can be gained in reliability of the scoring as there is an indication of a false negative rate at 16% with a larger training dataset. When clusters of events contain three reports, the confidence of those reports is high, and the system provides a satisfactory summary, however, this confidence decreases with a decline in the reports contained in those clusters.

6.6. Summary of Results

VerifiKar meets all five quantitative success criteria defined in the original project proposal:

Objective	Target	Achieved
Credibility scoring agreement with human labels	$\geq 70\%$	74%
Within-cluster > between-cluster similarity	Clear margin	0.81 vs 0.31
Summarisation factual accuracy (reviewer)	Acceptable majority	8/10 summaries
Full provenance links in all summaries	100%	10/10 summaries
Interactive API response times	< 2 seconds	All endpoints

7. Conclusion

7.1. Summary

VerifiKar clearly shows that a nine person team can build a production-grade AI pipeline within nine months to support filtered crowd sourced reporting. The pipeline collects reports from citizens using a real time mobile framework. The reports then go through the Ollama AI text clean and convert modules, the nomic AI text and CLIP image embedding, the Ateeqq AI versus human image detection, the pgvector cosine similarity clustering, and lastly the Ollama summarization. Once the posts have been converted the images and due to the fact that the pgvector process focuses on image and reporting on PostGIS and pgvector, which then delivers in real time through a Neon serverless SQL database.

All eight project objectives have been met. Verification reached a 74% agreement. The target was 70% agreement. The clustering module shows a systematic 0.81 similarity and 0.31 dissimilarity. The summarization reached a 8 out of 10 evaluations of factually (with AAAS) and full provenance in all 10. The system remains within the interaction load API response time. The full system is deployed and has a demo open to academic review.

7.2. Limitations

Limitation	Description
Data Quality Dependence	System performance relies heavily on user-submitted report quality. Poor or incomplete inputs reduce accuracy in verification, clustering, and post generation.
Moderate Credibility Agreement	74% accuracy and kappa of 0.48 indicate moderate, not high, agreement. The false negative rate (16%) means some manipulated content may pass the credibility filter.
Clustering Threshold Sensitivity	Cosine similarity and spatial radius parameters were tuned on a small development dataset and may require adjustment for different event types or geographic densities.

Limitation	Description
Summarisation Drift	Ollama occasionally abstracts beyond its source anchors, introducing minor factual drift. A stricter extractive-only mode could mitigate this at the cost of readability.
Regional and Language Scope	The system is optimised for Karachi and partially for Roman-Urdu. Extension to other Pakistani cities or languages would require additional normalisation tuning and localised OSM data.
Infrastructure Constraints	The Neon free-tier imposes connection and compute limits. At 200 concurrent users, response times begin to degrade; full-scale deployment would require a paid tier.
No Offline Mode	The system requires a continuous internet connection; no offline report drafting or cached feed is supported.

7.3. Future Work

Area	Description
Advanced AI Models	Replace nomic-embed-text with a larger multilingual embedding model to improve Roman-Urdu and regional dialect handling. Explore fine-tuned credibility classifiers trained on labelled Karachi-domain data to supersede the general-purpose Ateeq classifier.
Deepfake and Video Detection	Extend media verification to video content using temporal deepfake detection models, enabling full multimodal consistency checking beyond still images.
Temporal Event Lifecycle Modelling	Implement event lifecycle tracking - emergence, escalation, resolution, and archival - with explicit temporal decay for cluster centroid updates to reduce false merges.
Real-Time Streaming Architecture	Replace ARQ polling with a Kafka or WebSocket-based streaming pipeline to reduce end-to-end processing latency below one second for time-critical events.
Human Moderation Dashboard	Develop a web-based administrative dashboard for reviewing escalated reports, overriding automated credibility scores, and managing cluster merges.
Emergency Service Integration	Integrate with Karachi traffic authority, KWSB, and K-Electric data feeds to correlate citizen reports with official infrastructure status for richer credibility signals.
Geographic Expansion	Extend coverage to additional Pakistani cities by loading city-specific OSM polygon datasets and adapting the spatial radius parameters for different urban densities.

8. References

- [1] "FixMyStreet FAQ," FixMyStreet, 2024. [Online]. Available: <https://www.fixmystreet.com/faq>
- [2] M. Andrejevic, "Automating surveillance: The Citizen app and the privatization of safety," *Surveillance & Society*, vol. 18, no. 3, pp. 330-344, 2020.
- [3] S. Praharaj, F. T. Zahura, T. D. Chen, Y. Shen, L. Zeng, and J. L. Goodall, "Assessing trustworthiness of crowdsourced flood incident reports using Waze data: A Norfolk, Virginia case study," *Transportation Research Record*, vol. 2675, no. 12, pp. 650-662, 2021.
- [4] R. Zellers, A. Holtzman, H. Rashkin, Y. Bisk, A. Farhadi, F. Roesner, and Y. Choi, "Defending against neural fake news," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2019, pp. 9054-9065.
- [5] G. Chhikara, A. Sharma, V. Gurucharan, K. Ghosh, and A. Chakraborty, "LaMSUM: Amplifying voices against harassment through LLM guided extractive summarisation of user incident reports," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2024.
- [6] M. Farnaghi, Z. Ghaemi, and A. Mansourian, "Dynamic spatio-temporal tweet mining for event detection: A case study of Hurricane Florence," *International Journal of Disaster Risk Science*, vol. 11, pp. 378-393, 2020.
- [7] P. Mathews, C. Gray, L. Mitchell, G. T. Nguyen, and N. G. Bean, "SMERC: Social media event response clustering using textual and temporal information," in *Proc. IEEE Int. Conf. Big Data (BigData)*, 2018, pp. 2422-2431.

[8] Y. George, S. Karunasekera, A. Harwood, and K. H. Lim, "Real-time spatio-temporal event detection on geotagged social media," *Journal of Big Data*, vol. 8, article 91, 2021.